Jon Pulsipher - 6012 - A3, SortedArraySet Analysis.

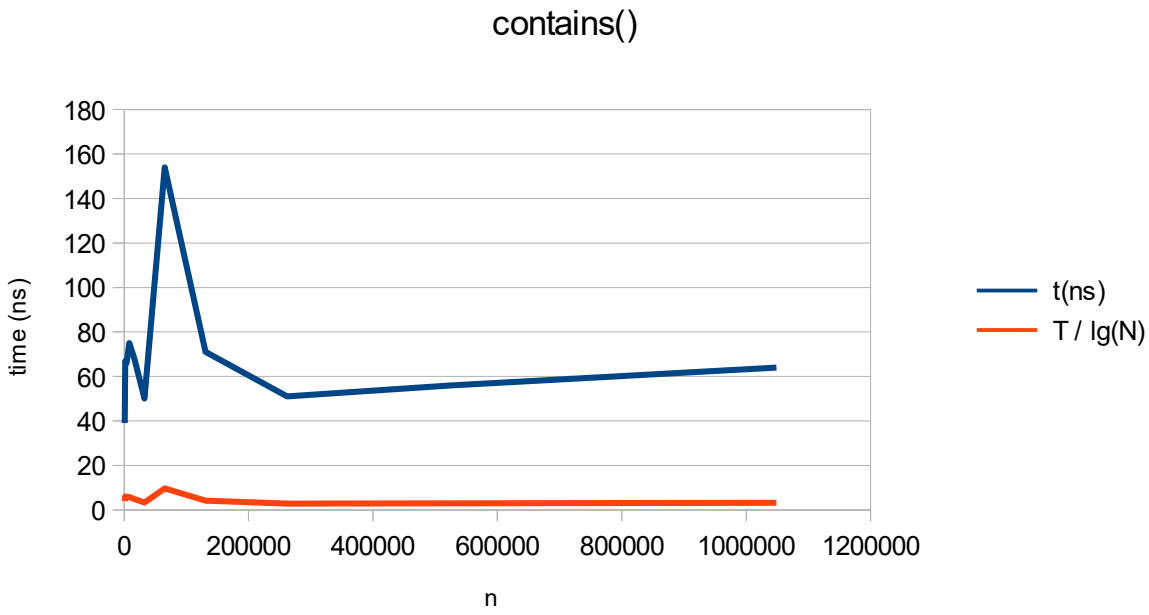
1) If you had backed the sorted set with a Java List instead of a basic array, summarize the main points in which your implementation would have differed. Do you expect that using a Java List would have more or less efficient and why? (Consider efficiency both in running time and in program development time.)

Assuming I used an ArrayList, I would have been able to let it handle re-sizing and shifting elements. I could have also let it handle addAll and removeAll, potentially with some performance improvements if it had any optimizations.

2) What do you expect the Big-O behavior of BinarySearchSet's contains method to be and why?

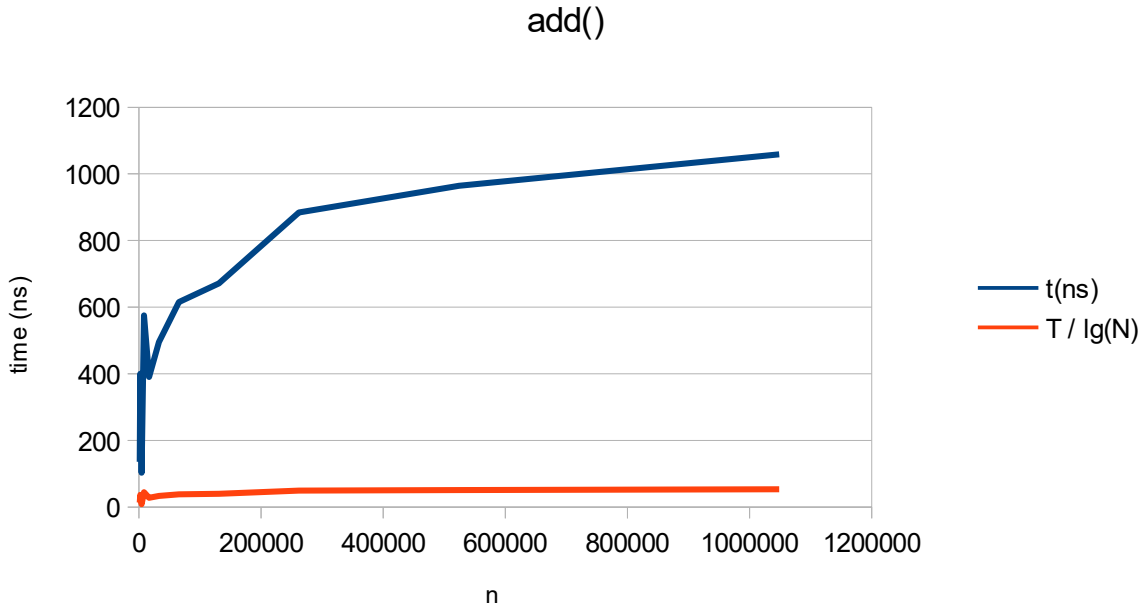
It implements the binary search algorithm which is O(lgN), so I expect it should be that. Why is that search algorithm O(lgN)? Because the list it searches halves each check, which gets small fast. How fast? O(lgN) fast.

3) Plot the running time of BinarySearchSet's contains method, using the timing techniques demonstrated in previous labs. Be sure to use a decent iteration count to get a reasonable average of running times. Include your plot in your analysis document. Does the growth rate of these running times match the Big-oh behavior you predicted in question 2?



This growth rate fits the expectation of O(lgN).The T/lg(n) graph is roughly constant.

4) Consider your add method. For an element not already contained in the set, how long does it take to locate the correct position at which to insert the element? Create a plot of running times. Pay close attention to the problem size for which you are collecting running times. Beware that if you simply add N items, the size of the sorted set is always changing. A good strategy is to fill a sorted set with N items and time how long it takes to add one additional item. To do this repeatedly (i.e., iteration count), remove the item and add it again, being careful not to include the time required to call remove() in your total. In the worst-case, how much time does it take to locate the position to add an element (give your answer using Big-oh)?



I've charted add() for arrays of 1 to N, adding a random number from 1 to N into the array 100 times. Unlike with contains, we are seeing growth for both t(n) and t(n)/lg(n). This is easier to see in the included table of data..

As for what the worst-case is for locating where to add an element, it is O(lgN), the same as contains(). In fact, they are literally the same in my implementation because contains() and the part of add() that locates the position use the same helper function.

add()		
n	t(ns)	T / lg(N)
1024	136	13.6
2048	400	36.363636364
4096	103	8.5833333333
8192	575	44.230769231
16384	390	27.857142857
32768	495	33
65536	615	38.4375
131072	671	39.470588235
262144	884	49.111111111
524288	964	50.736842105
1048576	1059	52.95